

```

in
(ht
tp
tp:
tp:
tp:
//t
wit
ter
.co
m/
sh
an
e_
a_
yn
n)

f
(ht
tp:
//f
ac
eb
oo
k.c
o
m/
m/
sh
an
an
erl
yn
n)

s:/
/w
w
w.l
ink
edi
n.c
o
m/
in/
sh
an
an
eal
yn
n)

in
(ht
tp:
tp:
tp:
//t
wit
ter
.co
m/
sh
an
e_
a_
yn
n)

```

Shane Lynn (<http://www.shanelynn.ie/>)

Data science, Startups, Analytics and Data visualisation.

Blog (<http://104.236.88.249>) About (<http://www.shanelynn.ie/about/>) Pandas Tutorials (<http://www.shanelynn.ie/category/tutorials/>)

Consultancy & Services (<http://www.shanelynn.ie/consultancy-services/>) Research (<http://www.shanelynn.ie/research/>)

Publications (<http://www.shanelynn.ie/publications/>) Contact (<http://www.shanelynn.ie/contact/>)

Oct 02 14 (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comments>)

Using iloc, loc, & ix to select rows and columns in Pandas DataFrames

Posted by shanelynn (<http://www.shanelynn.ie/author/shanelynn/>)



([http://www.shanelynn.ie/wp-](http://www.shanelynn.ie/wp-content/uploads/2016/10/Information-and-Data-Services.jpg)

[content/uploads/2016/10/Information-and-Data-Services.jpg](http://www.shanelynn.ie/wp-content/uploads/2016/10/Information-and-Data-Services.jpg))

Pandas Data Selection

There are multiple ways to select (<http://pandas.pydata.org/pandas-docs/stable/indexing.html#different-choices-for-indexing>) and index rows and columns from Pandas (<http://pandas.pydata.org/>) DataFrames (<http://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.html>). I find tutorials online focusing on advanced selections of row and column choices a little complex for my requirements.

Selection Options

There's three main options to achieve the selection and indexing activities in Pandas, which can be confusing. The three selection cases and methods covered in this post are:

1. Selecting data by row numbers (.iloc)
2. Selecting data by label or by a conditional statment (.loc)
3. Selecting in a hybrid approach (.ix) (now Deprecated in Pandas 0.20.1)

Data Setup

This blog post, inspired by other tutorials (<http://chrisalbon.com>), describes selection activities with these operations. The tutorial is suited for the general data science situation where, typically I find myself:

1. Each row in your data frame represents a data sample.
2. Each column is a variable, and is usually named. I rarely select columns without their names.
3. I need to quickly and often select relevant rows from the data frame for modelling and visualisation activities.

For the uninitiated, the Pandas (<http://pandas.pydata.org>) library for Python provides high-performance, easy-to-use data structures and data analysis tools for handling tabular data in "series" and in "data frames". It's brilliant at making your data processing easier and I've written before about grouping and summarising data (<http://104.236.88.249/summarising-aggregation-and-grouping-data-in-python-pandas/>) with Pandas.

Python Pandas Selections and Indexing

.iloc selections - position based selection

`data.iloc[<row selection>, <column selection>]`

Integer list of rows: [0,1,2]

Slice of rows: [4:7]

Single values: 1

Integer list of columns: [0,1,2]

Slice of columns: [4:7]

Single column selections: 1

loc selections - position based selection

`data.loc[<row selection>, <column selection>]`

Index/Label value: 'john'

List of labels: ['john', 'sarah']

Logical/Boolean index: data['age'] == 10

Named column: 'first_name'

List of column names: ['first_name', 'age']

Slice of columns: 'first_name':'address'

(<http://104.236.88.249/wp-content/uploads/2016/10/Pandas-selections-and-indexing.png>)

Summary of iloc and loc methods discussed in this blog post. iloc and loc are operations for retrieving data from Pandas dataframes.

Selection and Indexing Methods for Pandas DataFrames

For these explorations we'll need some sample data – I downloaded the uk-500 sample data set from www.briandunning.com (<https://www.briandunning.com/sample-data/>). This data contains artificial names, addresses, companies and phone numbers for fictitious UK characters. To follow along, you can download the .csv file here (<https://s3-eu-west-1.amazonaws.com/shanebucket/downloads/uk-500.csv>). Load the data as follows (the diagrams here come from a Jupyter notebook (<http://jupyter.org/>) in the Anaconda Python (<https://www.continuum.io/downloads>) install):

1	
2	<code>import pandas as pd</code>
3	<code>import random</code>
4	
5	<code># read the data from the downloaded CSV file.</code>
6	<code>data = pd.read_csv('https://s3-eu-west-1.amazonaws.com/shanebucket/downloads/uk-500.csv')</code>
7	<code># set a numeric id for use as an index for examples.</code>
8	<code>data['id'] = [random.randint(0,1000) for x in range(data.shape[0])]</code>
9	
10	<code>data.head(5)</code>
Pan view raw (https://gist.github.com/shanelynn/3324456b1a22eae86fce40bdd744f102/raw/2dd4ceaff99b0118fb72ac54fbc5630f66132b76/Pandas%20Index%20-%20Loading%20Data.py) das Index - Loading Data.py (https://gist.github.com/shanelynn/3324456b1a22eae86fce40bdd744f102#file-pandas-index-loading-data-py) hosted with ❤ by GitHub (https://github.com)	

In [124]: data.head(5)

Out[124]:

	first_name	last_name	company_name	address	city	county	postal	phone1	phone2	email	web
0	Aleshia	Tomkiewicz	Rosenburg Cpa Pc	14 Taylor St	St. Stephens Ward	Kent	CT2 7PP	01835-703597	01944-369967	atomkiewicz@hotmail.com	http://www
1	Evan	Zigomalas	Cap Gemini America	5 Binney St	Abbey Ward	Buckinghamshire	HP11 2AX	01937-864715	01714-737668	evan.zigomalas@gmail.com	http://www
2	France	Andrade	Elliott, John W Esq	8 Moor Place	East Southbourne and Tuckton W	Bournemouth	BH6 3BE	01347-368222	01935-821636	france.andrade@hotmail.com	http://www
3	Ulysses	Mcwalters	McMahan, Ben L	505 Exeter Rd	Hawerby cum Beesby	Lincolnshire	DN36 5RP	01912-771311	01302-601380	ulysses@hotmail.com	http://www
4	Tyisha	Veness	Champagne Room	5396 Forth Street	Greets Green and Lyng Ward	West Midlands	B70 9DT	01547-429341	01290-367248	tyisha.veness@hotmail.com	http://www

(http://104.236.88.249/wp-content/uploads/2016/09/pandas_indexing_example-1.png)

Example data loaded from CSV file.

1. Selecting pandas data using “iloc”

The iloc (<http://pandas.pydata.org/pandas-docs/version/0.17.0/generated/pandas.DataFrame.iloc.html>) indexer for Pandas DataFrame is used for integer-location based indexing / selection (<http://pandas.pydata.org/pandas-docs/stable/indexing.html#selection-by-position>) by position.

The iloc indexer syntax is data.iloc[<row selection>, <column selection>], which is sure to be a source of confusion for R users. “iloc” in pandas is used to **select rows and columns by number**, in the order that they appear in the data frame. You can imagine that each row has a row number from 0 to the total rows (data.shape[0]) and iloc[] allows selections based on these numbers. The same applies for columns (ranging from 0 to data.shape[1])

There are two “arguments” to iloc – a row selector, and a column selector. For example:

1	# Single selections using iloc and DataFrame
2	# Rows:
3	data.iloc[0] # first row of data frame (Aleshia Tomkiewicz) - Note a Series data type output.
4	data.iloc[1] # second row of data frame (Evan Zigomalas)
5	data.iloc[-1] # last row of data frame (Mi Richan)
6	# Columns:
7	data.iloc[:,0] # first column of data frame (first_name)
8	data.iloc[:,1] # second column of data frame (last_name)
9	data.iloc[:,-1] # last column of data frame (id)

view raw (<https://gist.github.com/shanealynn/1efd0555a0a668b5f0e3f5fa5593c673/raw/c68229b34a17f2179b28a08a51d0b1fce9b80738/Pandas%20Index%20-%20Single%20iloc%20selections.py>)
Pandas Index - Single iloc selections.py (<https://gist.github.com/shanealynn/1efd0555a0a668b5f0e3f5fa5593c673#file-pandas-index-single-iloc-selections-py>) hosted with ❤ by GitHub (<https://github.com>)

Multiple columns and rows can be selected together using the .iloc indexer.

1	# Multiple row and column selections using iloc and DataFrame
2	data.iloc[0:5] # first five rows of dataframe
3	data.iloc[:, 0:2] # first two columns of data frame with all rows
4	data.iloc[[0,3,6,24], [0,5,6]] # 1st, 4th, 7th, 25th row + 1st 6th 7th columns.
5	data.iloc[0:5, 5:8] # first 5 rows and 5th, 6th, 7th columns of data frame (county -> phone1).

view raw (<https://gist.github.com/shanealynn/3cd4c410ca1514dd9575443ab1b08a06/raw/ef0b34bed66d27ad60cb3960cd688b88069849d9/Pandas%20Index%20-%20Multi%20iloc%20selections.py>)
Pandas Index - Multi iloc selections.py (<https://gist.github.com/shanealynn/3cd4c410ca1514dd9575443ab1b08a06#file-pandas-index-multi-iloc-selections-py>) hosted with ❤ by GitHub (<https://github.com>)

There's two gotchas to remember when using iloc in this manner:

1. Note that .iloc returns a Pandas Series when one row is selected, and a Pandas DataFrame when multiple rows are selected, or if any column in full is selected. To counter this, pass a single-valued list if you require DataFrame output.

```
In [65]: print type(data.iloc[100])           # result of type series because only one row selected
          print type(data.iloc[[100]])       # result of type DataFrame because list selection used
          print type(data.iloc[2:10])        # result of type dataframe since there are two rows selected
          print type(data.iloc[1:2, 3])      # Series result because only one column selected
          print type(data.iloc[1:2, [3]])    # DataFrame result with one column be only one column selected
          print type(data.iloc[1:2, 3:6])    # DataFrame results because multiple columns and multiple rows.

<class 'pandas.core.series.Series'>
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.frame.DataFrame'>
```

(http://104.236.88.249/wp-content/uploads/2016/10/returning_dataframe_or_series_from_pandas_selections.png)

When using .loc, or .iloc, you can control the output format by passing lists or single values to the selectors.

2. When selecting multiple columns or multiple rows in this manner, remember that in your selection e.g.[1:5], the rows/columns selected will run from the first number to *one minus* the second number. e.g. [1:5] will go 1,2,3,4., [x,y] goes from x to y-1.

In practice, I rarely use the iloc indexer, unless I want the first (.iloc[0]) or the last (.iloc[-1]) row of the data frame.

2. Selecting pandas data using “loc”

The Pandas loc (<http://pandas.pydata.org/pandas-docs/version/0.17.0/generated/pandas.DataFrame.loc.html>) indexer can be used with DataFrames for two different use cases:

- a.) Selecting rows by label/index (<http://pandas.pydata.org/pandas-docs/stable/indexing.html#selection-by-label>)
- b.) Selecting rows with a boolean / conditional lookup (<http://pandas.pydata.org/pandas-docs/stable/indexing.html#boolean-indexing>)

The loc indexer is used with the same syntax as iloc: data.loc[<row selection>, <column selection>] .

2a. Label-based / Index-based indexing using .loc

Selections using the loc method are based on the index of the data frame (if any). Where the index is set on a DataFrame, using `df.set_index()`, the .loc method directly selects based on index values of any rows. For example, setting the index of our test data frame to the persons “last_name”:

1

data.set_index("last_name", inplace=True)

2

data.head()

view raw (<https://gist.github.com/shanealynn/12069e026b30f15d043193bdc5032846/raw/b84289c9399284b130df950aa23713733e027cf2/Pandas%20Index%20-%20Setting%20index%20for%20iloc.py>)

Pandas Index - Setting index for iloc.py (<https://gist.github.com/shanealynn/12069e026b30f15d043193bdc5032846#file-pandas-index-setting-index-for-iloc-py>) hosted with ❤ by GitHub (<https://github.com>)

In [10]: data.head()

Out[10]:

	first_name	company_name	address	city	county	postal	phone1	phone2	email
last_name									
Tomkiewicz	Aleshia	Alan D Rosenburg Cpa Pc	14 Taylor St	St. Stephens Ward	Kent	CT2 7PP	01835- 703597	01944- 369967	atomkiewicz@hotmail.com
Zigomalas	Evan	Cap Gemini America	5 Binney St	Abbey Ward	Buckinghamshire	HP11 2AX	01937- 864715	01714- 737668	evan.zigomalas@gmail.com
Andrade	France	Elliott, John W Esq	8 Moor Place	East Southbourne and Tuckton W	Bournemouth	BH6 3BE	01347- 368222	01935- 821636	france.andrade@hotmail.com
Mcwalters	Ulysses	Mcmahan, Ben L	505 Exeter Rd	Hawerby cum Beesby	Lincolnshire	DN36 5RP	01912- 771311	01302- 601380	ulysses@hotmail.com
Veness	Tyisha	Champagne Room	5396 Forth Street	Greets Green and Lyng Ward	West Midlands	B70 9DT	01547- 429341	01290- 367248	tyisha.veness@hotmail.com

(http://104.236.88.249/wp-content/uploads/2016/09/index_set_dataframe.png)

Last Name set as Index set on sample data frame

Now with the index set, we can directly select rows for different “last_name” values using .loc[<label>] – either singly, or in multiples. For example:

```
In [11]: data.loc['Andrade']

Out[11]: first_name      France
company_name      Elliott, John W Esq
address      8 Moor Place
city      East Southbourne and Tuckton W
county      Bournemouth
postal      BH6 3BE
phone1      01347-368222
phone2      01935-821636
email      france.andrade@hotmail.com
web      http://www.elliottjohnwesq.co.uk
id      487
Name: Andrade, dtype: object
```

(<http://104.236.88.249/wp-content/uploads/2016/09/pandas-loc-indexing-based-on-index-labels.png>)

Selecting single or multiple rows using .loc index selections with pandas. Note that the first example returns a series, and the second returns a DataFrame. You can achieve a single-column DataFrame by passing a single-element list to the .loc operation.

Select columns with .loc using the names of the columns. In most of my data work, typically I have named columns, and use these named selections.

```
In [46]: data.loc[['Andrade', 'Veness'], ['first_name', 'address', 'city']]
```

```
Out[46]:
```

	first_name	address	city
last_name			
Andrade	France	8 Moor Place	East Southbourne and Tuckton W

(http://104.236.88.249/wp-content/uploads/2016/10/named_column_selection_loc.png)

When using the .loc indexer, columns are referred to by names using lists of strings, or ":" slices.

You can select ranges of index labels – the selection `data.loc['Bruch':'Julio']` will return all rows in the data frame between the index entries for "Bruch" and "Julio". The following examples should now make sense:

```
1
2 # Select rows with index values 'Andrade' and 'Veness', with all columns between 'city' and 'email'
3 data.loc[['Andrade', 'Veness'], 'city':'email']
4 # Select same rows, with just 'first_name', 'address' and 'city' columns
5 data.loc[['Andrade':'Veness'], ['first_name', 'address', 'city']]
6
7 # Change the index to be based on the 'id' column
8 data.set_index('id', inplace=True)
9 # select the row with 'id' = 487
10 data.loc[487]
```

view raw (<https://gist.github.com/shanealynn/b27fbdce4688f06108a32c767b65fc3f/raw/c9020c045d7e2ccf5a6201b537423912b52cf350/Pandas%20Index%20-%20Select%20rows%20with%20loc.py>)
Pandas Index - Select rows with loc.py (<https://gist.github.com/shanealynn/b27fbdce4688f06108a32c767b65fc3f#file-pandas-index-select-rows-with-loc-py>) hosted with ❤ by GitHub (<https://github.com>)

Note that in the last example, `data.loc[487]` (the row with index value 487) is not equal to `data.iloc[487]` (the 487th row in the data). The index of the DataFrame can be out of numeric order, and/or a string or multi-value.

2b. Boolean / Logical indexing using .loc

Conditional selections with boolean arrays (<http://pandas.pydata.org/pandas-docs/stable/indexing.html#boolean-indexing>) using `data.loc[<selection>]` is the most common method that I use with Pandas DataFrames. With boolean indexing or logical selection, you pass an array or Series (<http://pandas.pydata.org/pandas-docs/stable/generated/pandas.Series.html>) of True/False values to the .loc indexer to select the rows where your Series has True values.

In most use cases, you will make selections based on the values of different columns in your data set.

For example, the statement `data['first_name'] == 'Antonio']` produces a Pandas Series with a True/False value for every row in the 'data' DataFrame, where there are "True" values for the rows where the first_name is "Antonio". These type of boolean arrays can be passed directly to the .loc indexer as so:

```
In [26]: data.loc[data['first_name'] == 'Antonio']
```

Out[26]:

	first_name	company_name	address	city	county	postal	phone1	phone2	email	web
last_name										
Villamarin	Antonio	Combs	353 Standish	Little Parndon	Hertfordshire	CM20	01559-	01388-	antonio.villamarin@gmail.com	http://www.com

(<http://104.236.88.249/wp-content/uploads/2016/10/Boolean-Selection-with-loc.png>)

Using a boolean True/False series to select rows in a pandas data frame – all rows with first name of "Antonio" are selected.

As before, a second argument can be passed to .loc to select particular columns out of the data frame. Again, columns are referred to by name for the loc indexer and can be a single string, a list of columns, or a slice ":" operation.

```
In [32]: data.loc[data['first_name'] == 'Erasmus', ['company_name', 'email', 'phone1']]
```

Out[32]:

	company_name	email	phone1
last_name			
Talentino	Active Air Systems	erasmo.talentino@hotmail.com	01492-454455
Gath	Pan Optx	egath@hotmail.com	01445-796544
Rhea	Martin Morrissey	erasmo_rhea@hotmail.com	01507-386397

(http://104.236.88.249/wp-content/uploads/2016/10/multiple_column_select_loc_indexer.png)

Selecting multiple columns with loc can be achieved by passing column names to the second argument of .loc[]

Note that when selecting columns, if one column only is selected, the .loc operator returns a Series. For a single column DataFrame, use a one-element list to keep the DataFrame format, for example:

(http://104.236.88.249/wp-content/uploads/2016/10/loc_indexer_returns_series_or_dataframe.png)

If selections of a single column are made as a string, a series is returned from .loc. Pass a list to get a DataFrame back.

Make sure you understand the following additional examples of .loc selections for clarity:

1	
2	# Select rows with first name Antonio, # and all columns between 'city' and 'email'
3	data.loc[data['first_name'] == 'Antonio', 'city':'email']
4	
5	# Select rows where the email column ends with 'hotmail.com', include all columns
6	data.loc[data['email'].str.endswith("hotmail.com")]
7	
8	# Select rows with last_name equal to some values, all columns
9	data.loc[data['first_name'].isin(['France', 'Tyisha', 'Eric'])]
10	
11	# Select rows with first name Antonio AND hotmail email addresses
12	data.loc[data['email'].str.endswith("gmail.com") & (data['first_name'] == 'Antonio')]
13	
14	# select rows with id column between 100 and 200, and just return 'postal' and 'web' columns
15	data.loc[(data['id'] > 100) & (data['id'] <= 200), ['postal', 'web']]
16	
17	# A lambda function that yields True/False values can also be used.
18	# Select rows where the company name has 4 words in it.
19	data.loc[data['company_name'].apply(lambda x: len(x.split(' ')) == 4)]
20	
21	# Selections can be achieved outside of the main .loc for clarity:
22	# Form a separate variable with your selections:
23	idx = data['company_name'].apply(lambda x: len(x.split(' ')) == 4)
24	# Select only the True values in 'idx' and only the 3 columns specified:
25	data.loc[idx, ['email', 'first_name', 'company']]

```
view raw (https://gist.github.com/shanealynn/b34acd07fdaba4f220f4e03c6f902a9f/raw/565aa819a584b4dafb0f9d6e9cbf40a228152e05/Pandas%20index%20-%20loc%20selection%20examples.py)
Pandas index - loc selection examples.py (https://gist.github.com/shanealynn/b34acd07fdaba4f220f4e03c6f902a9f#file-pandas-index-loc-selection-examples-py) hosted with ❤ by GitHub (https://github.com)
```

Logical selections and boolean Series can also be passed to the generic `[]` indexer of a pandas DataFrame and will give the same results: `data.loc[data['id'] == 9] == data[data['id'] == 9]`.

3. Selecting pandas data using ix

***Note:** The `ix` indexer has been deprecated (<http://pandas.pydata.org/pandas-docs/stable/whatsnew.html#whatsnew-0200-api-breaking-deprecate-ix>) in recent versions of Pandas, starting with version 0.20.1.*

The `ix[]` indexer (<http://pandas.pydata.org/pandas-docs/version/0.17.0/generated/pandas.DataFrame.ix.html>) is a hybrid of `.loc` and `.iloc`. Generally, `ix` is label based and acts just as the `.loc` indexer. However, `ix` also supports integer type selections (as in `.iloc`) where passed an integer. This only works where the index of the DataFrame is not integer based. `ix` will accept any of the inputs of `.loc` and `.iloc`.

Slightly more complex, I prefer to explicitly use `.iloc` and `.loc` to avoid unexpected results.

As an example:

```
1
2 # ix indexing works just the same as .loc when passed strings
3 data.ix[['Andrade']] == data.loc[['Andrade']]
4 # ix indexing works the same as .iloc when passed integers.
5 data.ix[[33]] == data.iloc[[33]]
6
7 # ix only works in both modes when the index of the DataFrame is NOT an integer itself.

Pandas view raw (https://gist.github.com/shanealynn/2c61235a30cbe1dff15d81b90c216cfe/raw/5e31330041dcecffa4f4552a86aa46b5d427bf0/Pandas%20index%20-%20ix%20selections.py)
index - ix selections.py (https://gist.github.com/shanealynn/2c61235a30cbe1dff15d81b90c216cfe#file-pandas-index-ix-selections-py) hosted with ❤ by GitHub (https://github.com)
```

Setting values in DataFrames using .loc

With a slight change of syntax, you can actually update your DataFrame in the same statement as you select and filter using `.loc` indexer. This particular pattern allows you to update values in columns depending on different conditions. The setting operation does not make a copy of the data frame, but edits the original data.

As an example:

```
1 # Change the first name of all rows with an ID greater than 2000 to "John"
2 data.loc[data['id'] > 2000, "first_name"] = "John"
3
4 # Change the first name of all rows with an ID greater than 2000 to "John"
5 data.loc[data['id'] > 2000, "first_name"] = "John"

view raw (https://gist.github.com/shanealynn/1c30da11d88cba2ecd3936209a981b9/raw/dc82d5f8b119343bfe17849f4ac97aebd82b5eb0/Pandas%20index%20-%20changing%20data%20with%20loc.py)
Pandas index - changing data with loc.py (https://gist.github.com/shanealynn/1c30da11d88cba2ecd3936209a981b9#file-pandas-index-changing-data-with-loc-py) hosted with ❤ by GitHub (https://github.com)
```

That's the basics of indexing and selecting with Pandas. If you're looking for more, take a look at the `.iat`, (<http://pandas.pydata.org/pandas-docs/version/0.17.0/generated/pandas.DataFrame.iat.html>) and `.at` (<http://pandas.pydata.org/pandas-docs/version/0.17.0/generated/pandas.DataFrame.at.html>) operations for some more performance-enhanced value accessors (<http://pandas.pydata.org/pandas-docs/stable/indexing.html#fast-scalar-value-getting-and-setting>) in the Pandas Documentation (<http://pandas.pydata.org/pandas-docs/stable/>) and take a look at selecting by callable functions (<http://pandas.pydata.org/pandas-docs/stable/indexing.html#indexing-callable>) for more `iloc` and `loc` fun.

📄 [blog](http://www.shanelynn.ie/category/blog/) (<http://www.shanelynn.ie/category/blog/>), [data science](http://www.shanelynn.ie/category/blog/data-science/) (<http://www.shanelynn.ie/category/blog/data-science/>), [Pandas](http://www.shanelynn.ie/category/blog/pandas/) (<http://www.shanelynn.ie/category/pandas/>), [python](http://www.shanelynn.ie/category/blog/python/) (<http://www.shanelynn.ie/category/blog/python/>), [Tutorials](http://www.shanelynn.ie/category/tutorials/) (<http://www.shanelynn.ie/category/tutorials/>)

🔗 [code](http://www.shanelynn.ie/tag/code/) (<http://www.shanelynn.ie/tag/code/>), [data](http://www.shanelynn.ie/tag/data/) (<http://www.shanelynn.ie/tag/data/>), [data science](http://www.shanelynn.ie/tag/data-science/) (<http://www.shanelynn.ie/tag/data-science/>), [datascience](http://www.shanelynn.ie/tag/datascience/) (<http://www.shanelynn.ie/tag/datascience/>), [learning](http://www.shanelynn.ie/tag/learning/) (<http://www.shanelynn.ie/tag/learning/>), [pandas](http://www.shanelynn.ie/tag/pandas/) (<http://www.shanelynn.ie/tag/pandas/>), [python](http://www.shanelynn.ie/tag/python/) (<http://www.shanelynn.ie/tag/python/>), [tutorial](http://www.shanelynn.ie/tag/tutorial/) (<http://www.shanelynn.ie/tag/tutorial/>)

14 thoughts on “Using iloc, loc, & ix to select rows and columns in Pandas DataFrames”

Jay Jay (@TheValleyRock) (<https://twitter.com/TheValleyRock>)
January 21, 2017 at 8:40 pm (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-401>)

Thanks for going straight to the point.

S C
Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=436#respond>)
March 16, 2017 at 11:07 pm (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-436>)

Part 1:

`data.iloc[:,1]` # second row of data frame (last_name)

`data.iloc[:, -1]` # last row of data frame (id)

row should be column?

o

shanelynn
Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=437#respond>)
March 16, 2017 at 11:37 pm (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-437>)

You're absolutely correct Shuca. I'll fix that ASAP and thanks for the spot!

Pingback: Pandas Notes | wallstbullean (<https://wallstbullean.wordpress.com/2017/03/19/pandas-notes/>)

sujithpadaru (@sujithpadaru) (<https://twitter.com/sujithpadaru>)
Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=442#respond>)
March 27, 2017 at 2:17 pm (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-442>)

It was really helpful!

o

shanelynn
Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=444#respond>)
March 28, 2017 at 8:33 am (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-444>)

I'm glad you found it so Sujith! Thanks for the comment.

hedgehoginventions (<http://hedgehoginventions.wordpress.com>)
Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=452#respond>)
April 6, 2017 at 11:33 am (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-452>)

Awesome post! I was struggling with pandas for a few days already and then I found your post, it really is the missing manual for pandas selectors that helped me understand it all. Much appreciated!

Liza K
Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=458#respond>)
April 22, 2017 at 9:15 am (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-458>)

Hi there! Much easier to understand than the pandas documentation.

I've got a question though, the answer to which eludes me despite trying to get through the documentation and hours of searching.

Is it possible, using `.loc` to exclude the line numbers and column headings from the output?

Cheers!

o

shanelynn
Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=459#respond>)
April 22, 2017 at 9:34 am (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-459>)

Hi Lisa. Glad you found it useful. Pandas data frames usually have column headers. You can reset indices using `reset_index` and maybe get rid of columns by changing to a numpy matrix using `as_matrix` I think.

arkojyoti2016 (<http://arkojyoti2016.wordpress.com>)
Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=463#respond>)
May 7, 2017 at 5:01 pm (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-463>)

Hi Shane,

Thanks for the post. However, I have a small doubt. In the following line of code:

`data.iloc[[0,4,7,25], [0,5,6]]` # 1st, 4th, 7th, 25th row + 1st 6th 7th columns

Shouldn't it be:

```
data.iloc[[0,3,6,24], [0,5,6]] # 1st, 4th, 7th, 25th row + 1st 6th 7th columns
```

Considering that the indexing in Python starts from 0? Please correct my understanding on the same. Also, if you could also provide some more examples on .ix it would be great 😊

o

shanelynn Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=464#respond>)
May 8, 2017 at 9:52 am (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-464>)

Hi! You're spot on there – that's a massive misprint by me. I will update pronto! Thanks for spotting it.

As for ix – its mainly in the Pandas API for legacy reasons as I understand things – you are almost always better off being explicit and using iloc or loc for selection.

ToddpG Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=472#respond>)
May 18, 2017 at 10:02 pm (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-472>)

Great post, thanks for the examples.

Dave Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=506#respond>)
July 21, 2017 at 6:37 pm (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-506>)

Thanks for a clear presentation, Shane. Is there a way to index on two columns? E.g. in your example some entries might have the same last name, and therefore not unique. Could the index be both last and first names?

o

shanelynn Reply (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/?replytocom=509#respond>)
August 3, 2017 at 9:30 pm (<http://www.shanelynn.ie/select-pandas-dataframe-rows-and-columns-using-iloc-loc-and-ix/#comment-509>)

Hi Dave. Good question, and I probably should have mentioned more about this. You can definitely select based on two or more different columns using logical operators. For e.g.:

```
data.loc[(data['first_name'] == 'Shane') & (data['last_name'] == 'Lynn')]
```

will work well. Just make sure you have the parentheses ()!

Leave a Reply

Coffee money generator:

My Tweets

Archives

- May 2017 (<http://www.shanelynn.ie/2017/05/>)
- April 2017 (<http://www.shanelynn.ie/2017/04/>)
- March 2017 (<http://www.shanelynn.ie/2017/03/>)
- November 2016 (<http://www.shanelynn.ie/2016/11/>)
- October 2016 (<http://www.shanelynn.ie/2016/10/>)
- July 2016 (<http://www.shanelynn.ie/2016/07/>)
- April 2016 (<http://www.shanelynn.ie/2016/04/>)
- October 2015 (<http://www.shanelynn.ie/2015/10/>)
- June 2015 (<http://www.shanelynn.ie/2015/06/>)
- December 2014 (<http://www.shanelynn.ie/2014/12/>)
- July 2014 (<http://www.shanelynn.ie/2014/07/>)
- March 2014 (<http://www.shanelynn.ie/2014/03/>)
- February 2014 (<http://www.shanelynn.ie/2014/02/>)
- December 2013 (<http://www.shanelynn.ie/2013/12/>)
- November 2013 (<http://www.shanelynn.ie/2013/11/>)
- October 2013 (<http://www.shanelynn.ie/2013/10/>)
- September 2013 (<http://www.shanelynn.ie/2013/09/>)
- July 2013 (<http://www.shanelynn.ie/2013/07/>)

Subscribe to Blog via Email

Enter your email address to subscribe to this blog and receive notifications of new posts by email.

Subscribe

Categories

© 2016 ascent. All rights reserved (<http://wordpress.org/>) | Ascent by NetTantra (<http://nettantra.com/>)